

Heart Disease Risk Prediction

An End-to-End MLOps Pipeline

MLOps AIMLCZG523 — Assignment 01

Name:

[STUDENT: insert your name and BITS ID here]

Table of Contents

1. Project Overview

This project builds a binary classifier that predicts presence of heart disease from 13 patient features in the UCI Heart Disease (Cleveland) dataset, and operationalizes it as a monitored, containerized, CI/CD-deployed API — covering the full MLOps lifecycle from data acquisition through production deployment.

Problem statement: given patient health measurements (age, blood pressure, cholesterol, ECG results, etc.), predict the risk of heart disease and serve that prediction through a production-ready API.

2. Setup / Install Instructions

Full instructions are in README.md. Summary:

- `pip install -r requirements.txt`
- `python data/download_data.py`
- `python src/eda.py` & `python src/train.py`
- `uvicorn src.api.main:app --port 8000` (or: `docker build/run` — see README)
- `pytest tests/ -v`

3. Exploratory Data Analysis

The dataset contains 303 patients and 13 predictive features plus a 5-level target collapsed to binary (0 = no disease, 1 = disease present) per the assignment's binary-target requirement. Two features had missing values: `ca` (4 rows) and `thal` (2 rows).

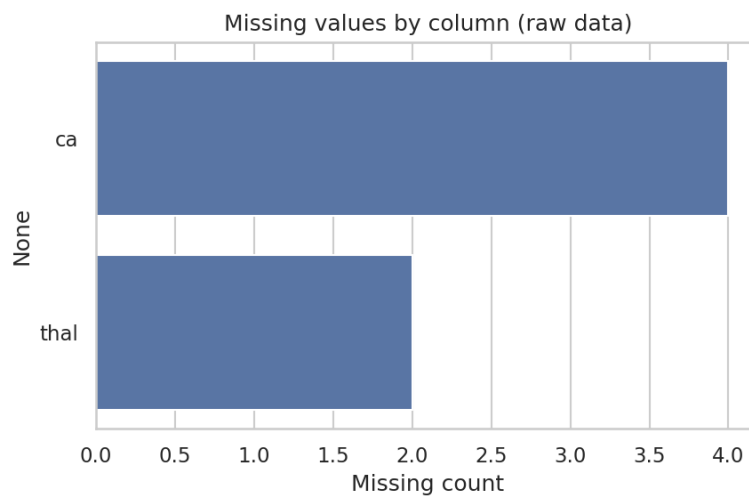


Figure 1. Missing values by column (raw data).

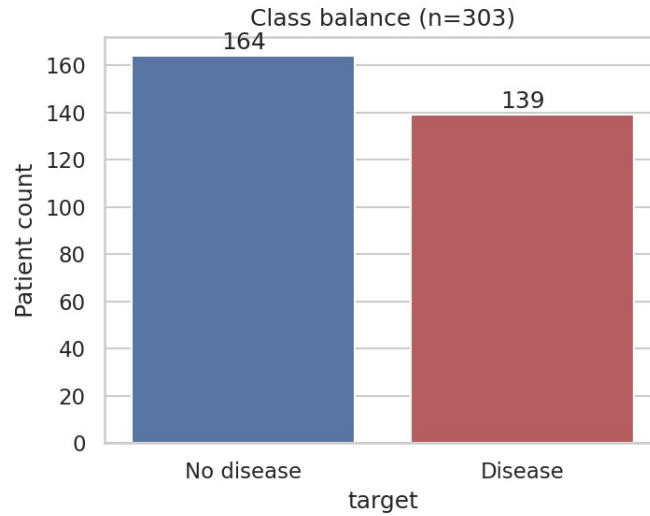


Figure 2. Class balance: 164 no-disease vs. 139 disease.

[STUDENT: describe in your own words whether you consider this class balance acceptable, and why you did or didn't apply class weighting / resampling.]

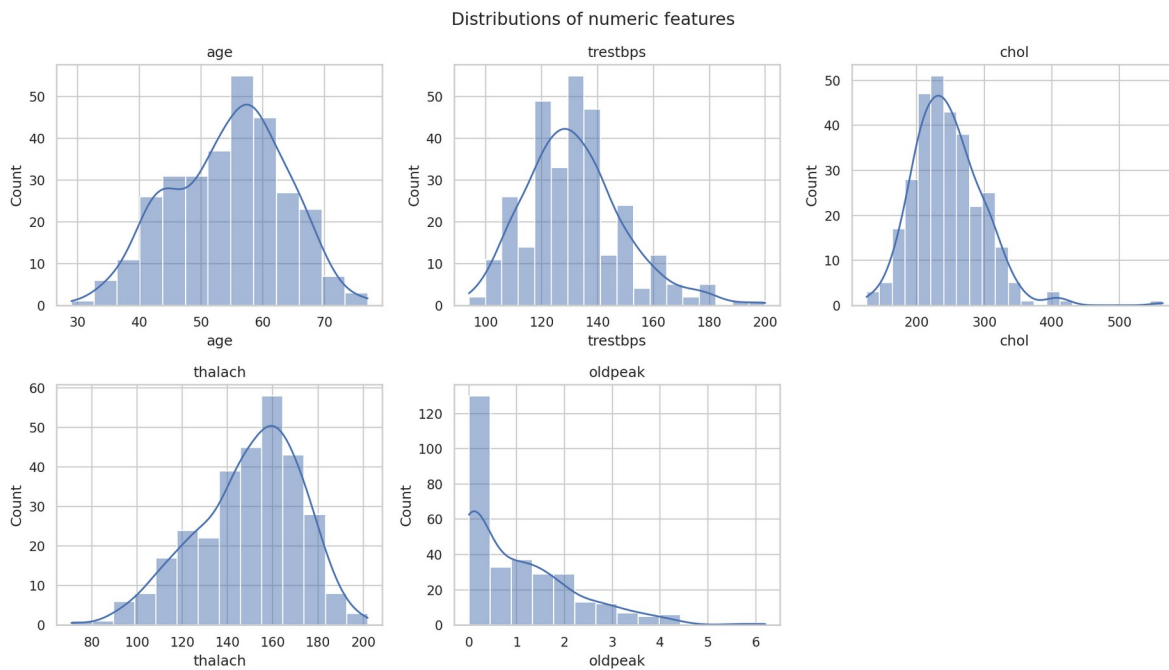


Figure 3. Distributions of numeric features.

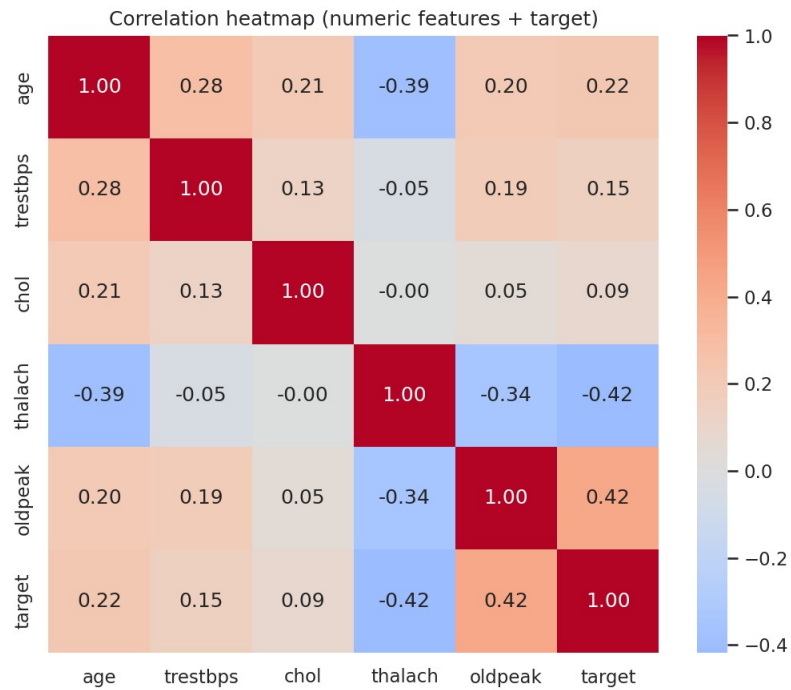


Figure 4. Correlation heatmap (numeric features + target).

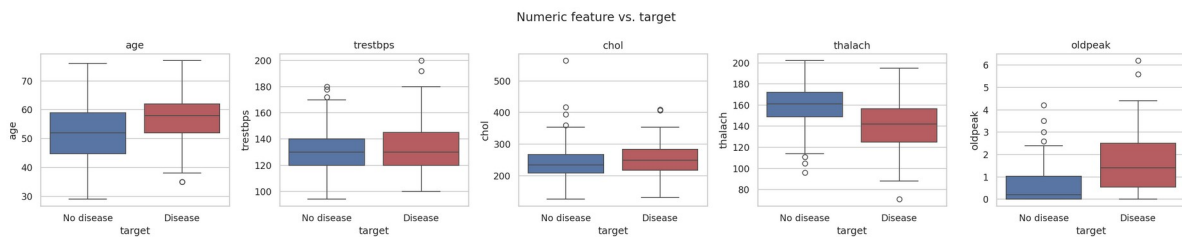


Figure 5. Numeric feature distributions split by target class.

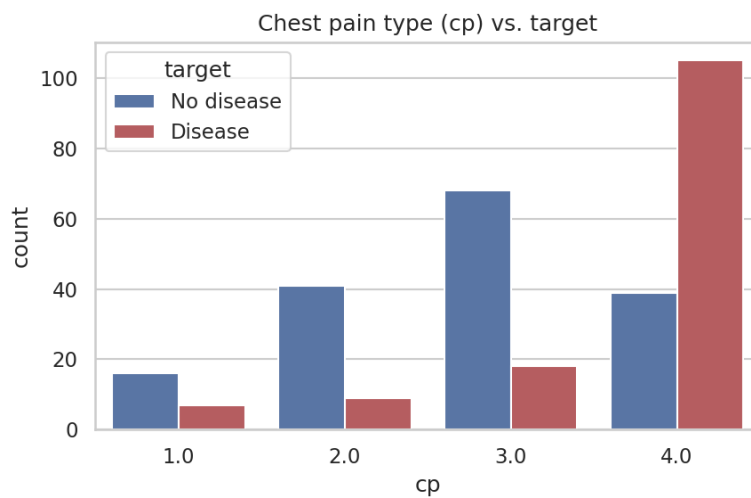


Figure 6. Chest pain type (cp) vs. target.

[STUDENT: add your own clinical/statistical interpretation of at least 2 of the figures above — which features look most separable by eye, and does that match the correlation heatmap?]

4. Feature Engineering & Model Development

Preprocessing: numeric features (age, trestbps, chol, thalach, oldpeak) are median-imputed and standard-scaled; categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal) are mode-imputed and one-hot encoded. Both live inside a single sklearn Pipeline together with the classifier, so the exact fitted transformer is what gets serialized and served — there is no separate preprocessing artifact to fall out of sync.

Three classifiers were trained and tuned via GridSearchCV (5-fold, scoring=roc_auc): Logistic Regression, Random Forest, and XGBoost.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV best ROC-AUC
Logistic Regression (best)	0.885	0.839	0.929	0.881	0.965	0.898
XGBoost	0.885	0.862	0.893	0.877	0.949	0.882
Random Forest	0.852	0.828	0.857	0.842	0.947	0.904

Logistic Regression was selected as the final model based on held-out test ROC-AUC (0.965). Notably, Random Forest had the highest cross-validation score (0.904) but the lowest test-set ROC-AUC (0.947) of the three — with only ~61 test rows, CV rank order and test rank order are not guaranteed to agree; this is sampling variance, not a modeling error.

[STUDENT: add your own discussion here: would you trust CV score or test score more for model selection on a dataset this small, and why? Would nested cross-validation change your answer?]

5. Experiment Tracking (MLflow)

All three models' parameters, metrics (accuracy, precision, recall, F1, ROC-AUC, CV best ROC-AUC), confusion matrices, ROC curves, and serialized pipelines are logged to MLflow under the experiment 'heart-disease-classification', using a SQLite-backed tracking store (sqlite:///mlflow.db).

[STUDENT: run `mlflow ui --backend-store-uri sqlite:///mlflow.db`, screenshot the experiment comparison view, and paste it here.]

6. Model Packaging & Reproducibility

The winning pipeline (preprocessing + classifier bundled together) is saved via joblib to models/model.joblib, alongside models/metadata.json recording which model won and its metrics. Dependencies are pinned in requirements.txt against versions verified by an actual clean-environment run (not just copied from a tutorial).

7. CI/CD Pipeline & Automated Testing

GitHub Actions (.github/workflows/ci-cd.yml) runs on every push/PR: lint (ruff) -> download data -> train -> EDA -> pytest -> upload model as a build artifact -> a second job downloads that artifact and builds + smoke-tests the Docker image. Any failing step halts the pipeline with a red status and that step's log, satisfying the 'must fail clearly on errors' requirement.

[STUDENT: screenshot a green (and, once, deliberately a red) GitHub Actions run and paste both here.]

8. Model Containerization

The FastAPI serving app exposes POST /predict (JSON in, prediction + probability + model name out), GET /health (readiness/liveness target), and GET /metrics (Prometheus scrape target). The Dockerfile bakes in the trained model and runs as a non-root user.

[STUDENT: screenshot `docker build` succeeding and a successful `curl /predict` against the running container, and paste both here.]

9. Production Deployment

Kubernetes manifests (k8s/deployment.yaml, k8s/service.yaml) define a 2-replica Deployment with readiness/liveness probes on /health, and a LoadBalancer Service. An optional Helm chart is provided under k8s/helm/heart-disease-api/.

[STUDENT: deploy to Minikube or your cloud account of choice, screenshot `kubectl get pods,svc`, and paste your access URL/instructions here.]

10. Monitoring & Logging

The API logs every prediction request/response to stdout. Prometheus scrapes /metrics (request rate, latency histogram, error rate); a docker-compose stack (monitoring/docker-compose.yml) runs the API + Prometheus + Grafana together for a fully local demo, decoupled from whichever deployment target was used above.

[STUDENT: screenshot your Grafana dashboard after sending it a few requests, and paste it here.]

11. Architecture

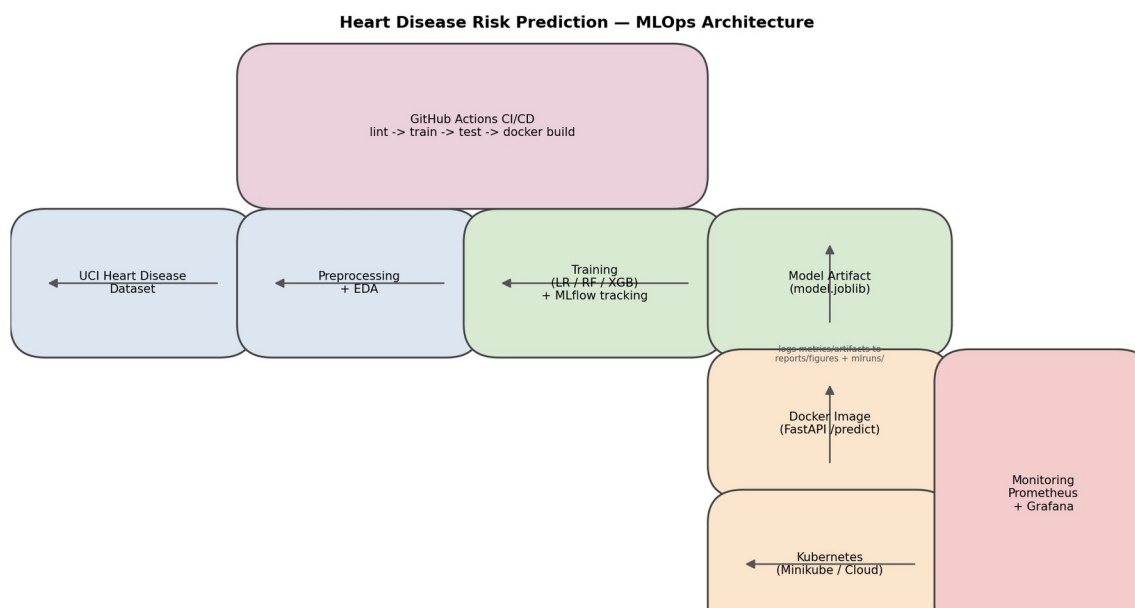


Figure 7. End-to-end architecture.

12. Repository & Video

[STUDENT: insert your GitHub repository URL here.]

[STUDENT: insert the link to your short video walkthrough here.]

13. Conclusion & Reflection

[STUDENT: write your own closing reflection: what was hardest, what you'd do differently with more time, and one thing you'd change about this pipeline if this were a real production system.]